

AI-DRIVEN BUG REPORT CLASSIFICATION AND SEVERITY PREDICTION SYSTEM

Course Name - CSI-5130/4130 - Artificial Intelligence - Winter-2026

Prof - Dr. Tianle Ma



Presented By -

- 1. Bhavana Battala*
- 2. Sai Deepthi K*
- 3. Priyanka Mirgule*
- 4. Shriya Deshpande*

Contents

- Introduction and Project Goals
- Problem Statement
- Flowchart
- Dataset Overview
- Dataprocessing
- Modelling Approach
- Comparative Analysis
- Limitations
- Conclusion and Future Work
- References
- Q&A



Introduction and Project Goals

- Software systems generate **large-scale bug reports** during development and deployment
- Bug reports contain **unstructured natural language text**, making them difficult to process automatically
- Effective bug triaging involves:
 - Identifying the type of issue (classification)
 - Determining its urgency (severity prediction)
- Traditional manual triaging:
 - Requires significant engineering effort
 - Is prone to human error and inconsistency
 - Does not scale with increasing project complexity



Problem Statement

Bug reports are:

- Noisy
- Inconsistent in structure
- Highly dependent on developer language and style

Key Challenges:

- Extract meaningful features from textual data
- Handle **multi-class classification** for bug categories
- Predict severity levels accurately
- Address **class imbalance** in real-world datasets

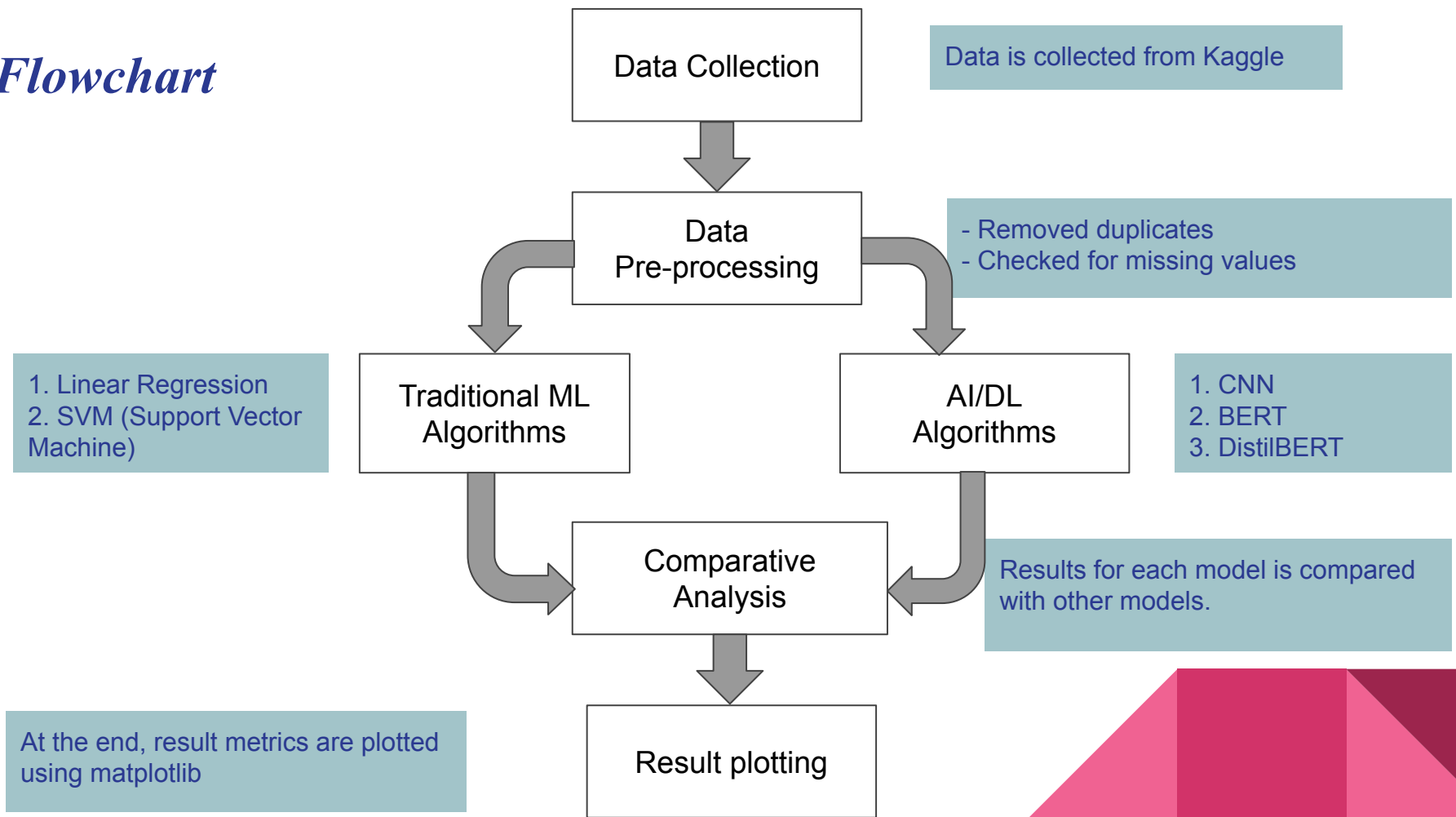
Core Problem

How to design a system that:

- Understands textual bug reports
- Classifies them correctly
- Predicts severity reliably



Flowchart



Dataset Overview

- ~50,000 real-world bug reports
- Source: Kaggle dataset

Data Includes

- **Title** → short summary (keywords like “crash”, “error”)
- **Description** → detailed bug information
- **Bug Category** → multi-class label (UI, performance, etc.)
- **Severity Level** → priority (low, medium, high)

Data Characteristics

- Unstructured textual data
- Multi-class classification problem
- Contains noise (typos, inconsistent wording)



Data Preprocessing

Preprocessing Steps

- **Data Cleaning**
 - Removed missing values and duplicates
- **Text Consolidation**
 - Combined *title* + *description* for richer context
- **Text Normalization**
 - Lowercasing (reduces vocabulary size)
 - Removed punctuation and non-alphanumeric characters
- **Stop-word Removal**
 - Removed low-information words to reduce noise
- **Label Encoding**
 - Converted bug categories and severity levels into numerical format



Modeling Approach

- This project evaluates **multiple models for bug classification and severity prediction**.
- Focus is on understanding how different models handle **text representation and context**.

Models Implemented

- Logistic Regression (Traditional ML)
- Support Vector Machine (Traditional ML)
- Convolutional Neural Network (Deep Learning)
- BERT (Transformer)
- DistilBERT (Optimized Transformer)

Model Progression

- Traditional ML → learns **word importance (frequency-based)**.
- Deep Learning → captures **patterns in sequences** of words.
- Transformers → understand **context and semantic** meaning across the text.



Logistic Regression

Logistic Regression implemented on **TF-IDF features**

Input: bug report text

Performance

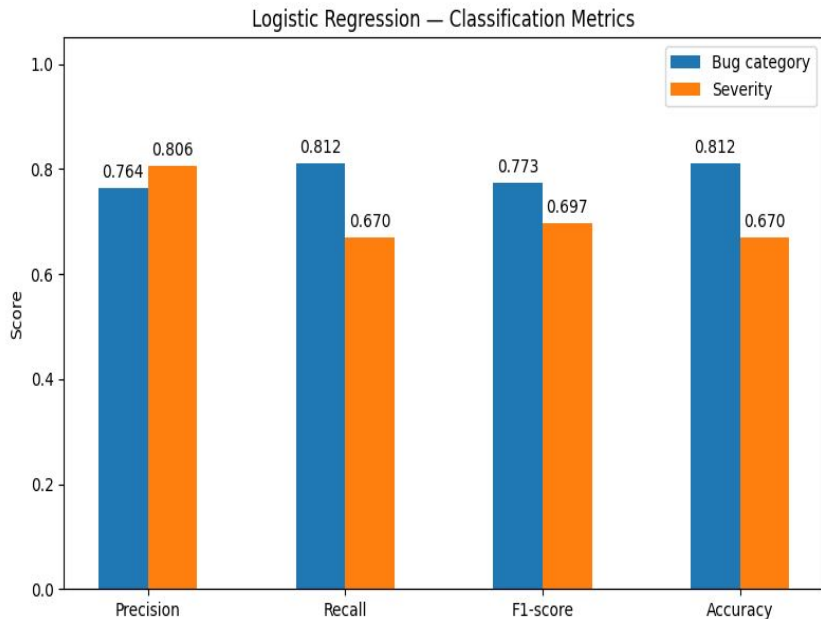
- Accuracy : **Bug-Category - 0.81 and Severity - 0.67**
- Served as a strong baseline model

Observation

- Performed consistently across classes
- Effectively captured **important keywords in bug reports**

Conclusion

- Provided strong baseline performance
- Limited by lack of contextual understanding of text
- Performance depends on TF-IDF representation



Support Vector Machine (SVM)

- Linear SVM implemented on **TF-IDF features**
- Input: bug report text

Performance

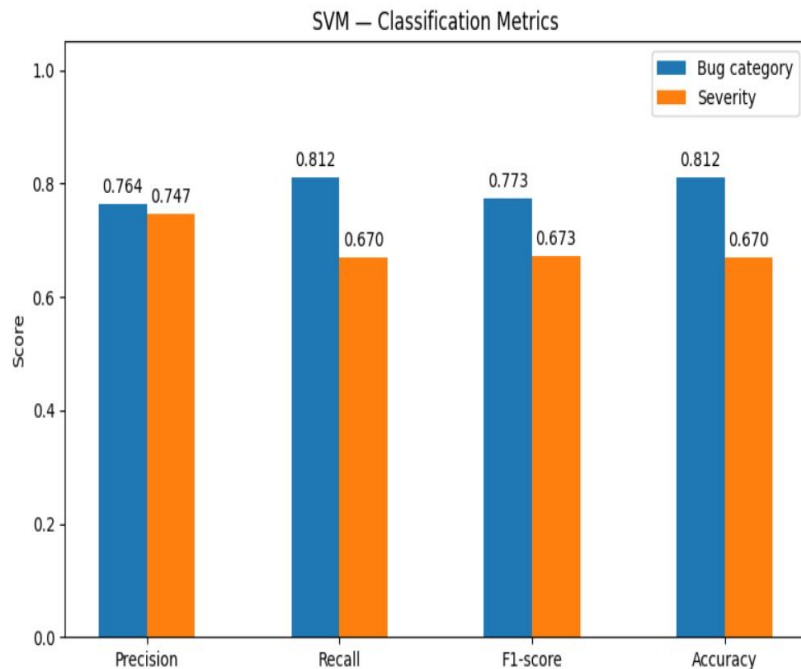
- Accuracy : **Bug-Category - 0.81** and **Severity - 0.67**
- Same as Logistic Regression

Observation

- No improvement over baseline
- Model relied on the same TF-IDF feature space

Conclusion

- SVM did not add value in this setup
- Feature representation limited performance



CNN (*Convolutional Neural Network*)

- Implemented using **Keras Sequential model** with tokenized input
- Input: padded sequences from bug report text

What it captures

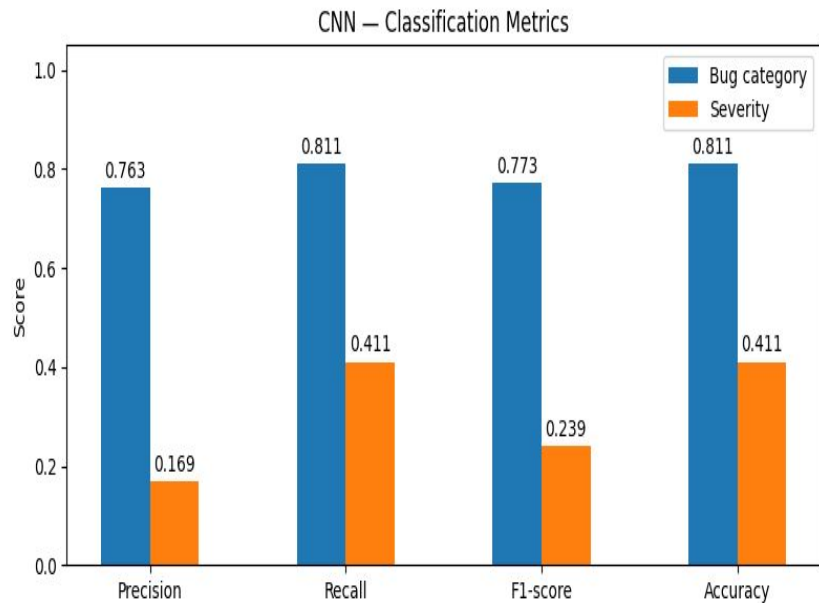
- Learns **local patterns and phrases**
- Identifies key expressions like:
 - “app crash”
 - “not responding”

Performance

- Accuracy : **Bug-Category - 0.81** and **Severity - 0.41**
- Comparable to TF-IDF models

Observation

- Improved over keyword-only learning
- But limited in capturing full context



BERT

Implemented using **bert-base-uncased (HuggingFace)**

Input: tokenized full bug report text

What it captures

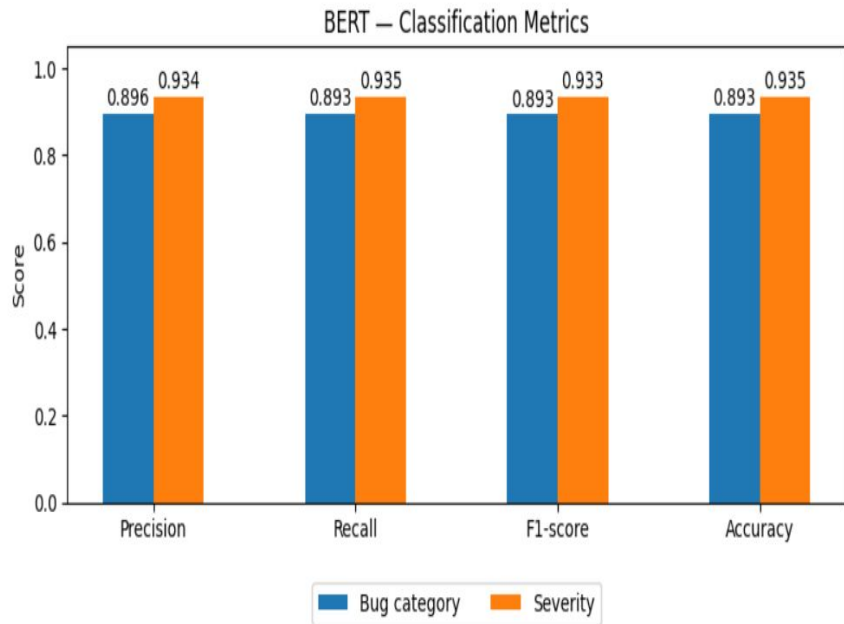
- Understands **context and relationships between words**
- Processes entire sentence using attention mechanism

Performance

- Accuracy : **Bug-Category - 0.89** and **Severity - 0.93**

Observation

- Lower than expected performance
- Likely impacted by:
 - Limited fine-tuning
 - Training setup



DistilBERT

Lightweight version of BERT with same pipeline

What it captures

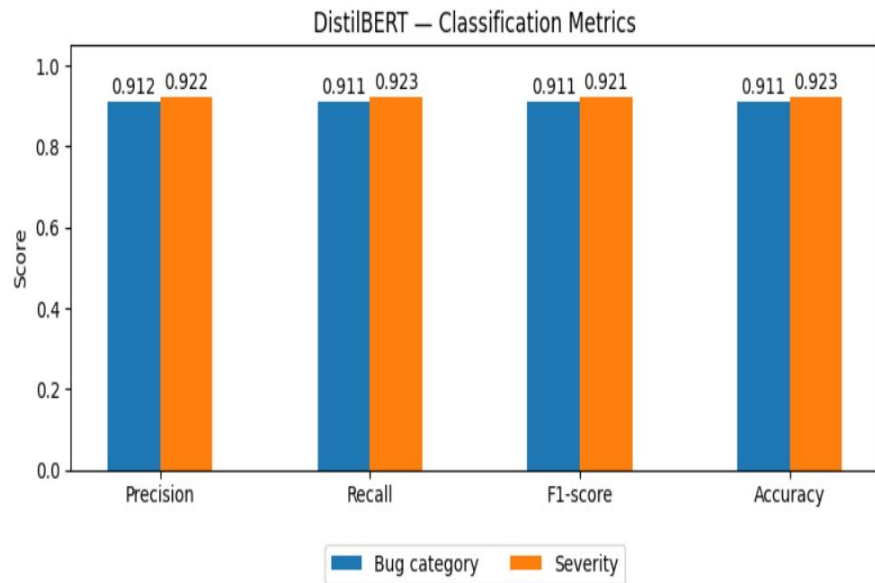
- Similar contextual understanding as BERT
- More efficient architecture

Performance

- Accuracy : **Bug-Category - 0.91 and Severity - 0.92**
- Better than BERT in this setup

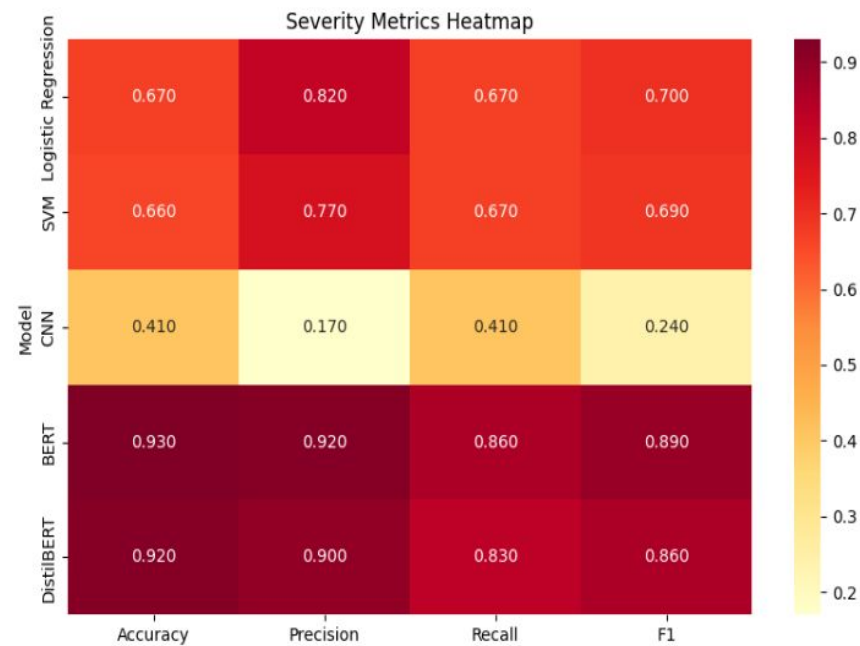
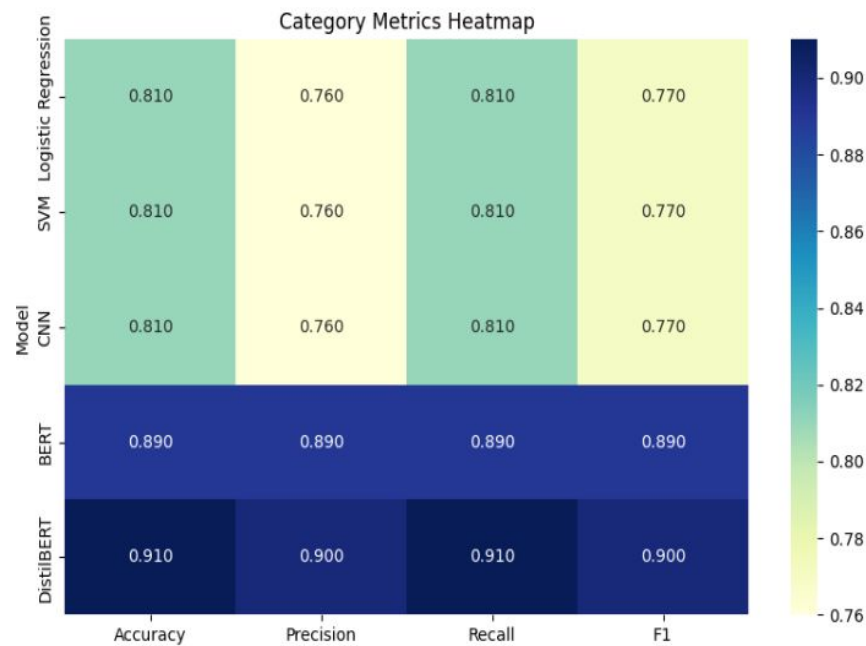
Observation

- Faster and more stable
- Better suited for limited training setup



Comparison of Model Performance Across Metrics

Model Comparison: Category & Severity (Heatmap)



<i>Model</i>	<i>Input Representation</i>	<i>What It Learns</i>	<i>Accuracy Category</i>	<i>Accuracy Severity</i>	<i>Key Observation</i>
Logistic Regression	TF-IDF	Keyword importance	~0.81	~0.67	Strong baseline
SVM	TF-IDF	Decision boundary	~0.81	~0.66	Comparable to LR
CNN	Tokenized sequences	Local patterns	~0.81	~0.41	Captures phrases
BERT	Contextual tokens	Full context	~0.89	~0.93	Best severity performance
DistilBERT	Contextual tokens	Context (efficient)	~0.91	~0.92	Best category performance

Key Insights

- **Traditional models (Logistic Regression, SVM):** ~0.81 accuracy for category, ~0.67 for severity; fast, interpretable, limited context understanding.
- **CNN:** Captures local patterns, similar category accuracy, slightly better for severity; struggles with long-range context.
- **Transformer models (BERT, DistilBERT):** Best at understanding context and semantics; improved performance for complex bug reports and severity prediction; DistilBERT faster but slightly less accurate.
- **Overall:** Model performance depends on features and complexity; transformer models excel at context and severity, while traditional models are fast and reliable for simpler cases.



Limitations

- **Dataset**
 - Class imbalance across categories
 - Noisy text (typos, inconsistent descriptions)
- **Traditional Models (LR, SVM)**
 - Depend on TF-IDF (no context understanding)
 - Limited to keyword-based learning
- **CNN**
 - Captures local patterns only
 - Limited long-range context understanding
- **BERT / DistilBERT**
 - Underperformed due to limited tuning
 - Requires more data and computational resources

Overall

- Performance highly dependent on preprocessing
 - Limited hyperparameter tuning
- 

Conclusion

- Bug classification can be effectively automated using ML models
- TF-IDF based models provided **strong and reliable performance**
- CNN added pattern learning but limited improvement
- DistilBERT performed better than BERT among transformers

Future Work

- Improve transformer performance through:
 - Better hyperparameter tuning
 - More training epochs
 - Larger or domain-specific datasets
 - Address class imbalance
 - Deploy model for real-time bug triaging
- 

References

1. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv:1810.04805.
2. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. arXiv:1910.01108.
3. Le, Q., & Mikolov, T. (2014). *Distributed Representations of Sentences and Documents*. Proceedings of ICML, 1188–1196.
4. Kim, Y. (2014). *Convolutional Neural Networks for Sentence Classification*. arXiv:1408.5882.
5. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
6. Yasir Abdullah Mirza, “50K Bug Dataset,” *Kaggle*, 2026. [Online]. Available: <https://www.kaggle.com/datasets/mirzayasirabdullah07/50k-bug-dataset?resource=download>

Thank You!!